

Lab 8 - Uso Consciente de LLMs e Prompt Engineering - André Hernandez

Parte 1 – Escolha da ferramenta

Nome da ferramenta utilizada: Gemini

Data de uso: 07/06/2026

Versão gratuita ou paga: gratuita

Se a ferramenta possuía acesso à internet: sim

Parte 2 – Primeira tarefa: explicação conceitual

1. Peça para a LLM explicar um conceito importante da Computação. Utilize inicialmente um prompt simples.

Prompt: “explique o conceito de Software “

Resposta:

O que é Software?

De forma direta, **software** é um conjunto de instruções, dados ou programas que dizem a um computador (ou celular, smartwatch, etc.) como ele deve funcionar e quais tarefas deve executar. Ele é a parte lógica e "invisível" da tecnologia.

A Metáfora: Hardware vs. Software

A maneira mais fácil de entender o conceito é contrastando-o com o seu parceiro inseparável, o **hardware**:

- **Hardware:** É tudo o que você pode tocar. São as peças físicas — a tela, o teclado, o processador, a placa-mãe. É o "corpo" da máquina.
- **Software:** É tudo o que você não pode tocar. São os códigos, aplicativos e sistemas. É a "mente" da máquina, dando propósito e direção às peças físicas.

Pense em um piano: O piano de madeira, as teclas e as cordas são o hardware. A partitura com as notas musicais — que diz exatamente quais teclas apertar e em qual momento — é o software. O piano não toca a música sozinho, e a partitura não produz som sem o piano.

Principais Tipos de Software

O mundo dos softwares é gigantesco, mas podemos dividi-lo em três categorias principais baseadas na função que desempenham:

1. **Software de Sistema:** É a base estrutural. Ele gerencia o hardware e cria um ambiente para que os outros programas consigam funcionar sem problemas. Ele opera nos "bastidores".
 - *Exemplos:* Sistemas operacionais como Windows, macOS, Linux, Android e iOS.
2. **Software Aplicativo:** É a camada com a qual você interage diretamente. São os programas criados para resolver problemas específicos do usuário final, seja para produtividade, entretenimento ou comunicação.
 - *Exemplos:* Microsoft Word, WhatsApp, navegadores (Google Chrome, Safari), Netflix e jogos de videogame.
3. **Software de Programação:** São as ferramentas usadas pelos desenvolvedores e engenheiros de software para criar *novos* softwares.
 - *Exemplos:* Editores de código (como o Visual Studio Code), compiladores e linguagens de programação (como Python, Java ou C++).

Em sua essência, todo software começa como linhas de texto escritas por um humano (o código-fonte), que depois são traduzidas para a "linguagem de máquina" (zeros e uns) que o processador do computador consegue entender e executar.

Você gostaria de entender como um software é construído passo a passo, ou prefere focar em como algum tipo específico de aplicativo funciona?

2. Crie um prompt mais elaborado contendo contexto, público-alvo e formato desejado.

Prompt: “Explique o conceito de Software, no contexto de uma disciplina de Introdução à Ciência da Computação, voltando-se para estudantes iniciantes na computação. Quero a resposta no formato de um resumo. “

Resposta:

Resumo: O Conceito de Software na Ciência da Computação

Na Ciência da Computação, **software** é fundamentalmente definido como um conjunto estruturado de instruções, dados e algoritmos que orientam o comportamento e o funcionamento de um sistema computacional. Ele atua como a camada lógica que transforma o potencial de processamento da máquina em ações e resultados concretos.

Para estudantes iniciantes, o conceito pode ser compreendido através dos seguintes pilares:

1. A Dualidade Hardware vs. Software

Enquanto o **hardware** compreende os componentes físicos e tangíveis (CPU, memória RAM, placa-mãe, periféricos), o **software** é a entidade intangível. A analogia clássica é tratar o hardware como o "corpo" da máquina e o software como a "mente" ou o "pensamento". O hardware por si só é inerte; é o software que lhe confere utilidade.

2. O Conceito de Abstração

Na computação, o software atua como uma **camada de abstração**. Isso significa que ele esconde a complexidade dos circuitos elétricos e das operações binárias (0s e 1s) que ocorrem nos bastidores, permitindo que usuários e programadores interajam com a máquina usando linguagens, interfaces e comandos muito mais próximos da linguagem humana e da lógica matemática.

3. Classificação Básica (As Três Camadas)

Do ponto de vista da arquitetura de sistemas, os softwares são geralmente divididos em três categorias hierárquicas:

- **Software de Sistema:** É a base. Interage diretamente com o hardware e fornece a plataforma para que outros programas funcionem.
 - *Exemplos:* Sistemas Operacionais (Linux, Windows, macOS) e drivers de dispositivos.
- **Software de Programação:** São as ferramentas que os cientistas da computação e desenvolvedores usam para criar, depurar e manter outros softwares. Eles traduzem a lógica humana para a linguagem da máquina.
 - *Exemplos:* Compiladores, interpretadores, depuradores (debuggers) e Ambientes de Desenvolvimento Integrado (IDEs).
- **Software Aplicativo:** É a camada superior, voltada para a realização de tarefas específicas para o usuário final.
 - *Exemplos:* Navegadores web, editores de texto, bancos de dados e jogos.

4. O Ciclo de Vida e Execução

Na prática acadêmica e profissional, o software começa como **código-fonte** — um texto escrito por programadores utilizando uma linguagem de programação (como C, Python ou Java). Este código descreve uma sequência lógica de passos (um algoritmo). Antes que o computador possa executar essa tarefa, o software de programação traduz esse código-fonte para **código de máquina** (instruções binárias elementares) que a Unidade Central de Processamento (CPU) pode finalmente ler, decodificar e executar.

3. Compare as duas respostas obtidas.

Ambas as respostas foram úteis, explicando, de fato, o que é o conceito de software, todavia, a segunda foi mais concisa e direcionada para o contexto em que estou inserido, pois eu detalhei o prompt.

Parte 3 – Segunda tarefa: geração de código

Solicite à LLM a geração de um pequeno programa.

O relatório deve conter:

- prompt utilizado:

“Crie um programa que calcule o IMC de uma pessoa conforme dados da OMS. A entrada é caracterizada pelo sexo do indivíduo, bem como altura e peso.”

- código gerado:

```
Python

def calcular_imc(peso, altura):
    # O cálculo do IMC é o peso dividido pela altura ao quadrado
    imc = peso / (altura ** 2)
    return imc

def classificar_imc(imc):
    # Classificação padrão da OMS para adultos
    if imc < 18.5:
        return "Abaixo do peso"
    elif 18.5 <= imc < 24.9:
        return "Peso normal"
    elif 25.0 <= imc < 29.9:
        return "Sobrepeso"
    elif 30.0 <= imc < 34.9:
        return "Obesidade Grau I"
    elif 35.0 <= imc < 39.9:
        return "Obesidade Grau II"
    else:
        return "Obesidade Grau III"

# Coleta de dados do usuário
print("=== Calculadora de IMC (Padrão OMS) ===")
sexo = input("Digite o sexo (M/F): ").upper().strip()
peso = float(input("Digite o peso em kg (ex: 70.5): "))
altura = float(input("Digite a altura em metros (ex: 1.75): "))

# Execução do cálculo e classificação
imc = calcular_imc(peso, altura)
classificacao = classificar_imc(imc)

# Exibição dos resultados
print("\n--- Resultados ---")
if sexo == 'M':
    print("Sexo: Masculino")
elif sexo == 'F':
    print("Sexo: Feminino")
else:
    print("Sexo: Não especificado ou outro")

print(f"Seu IMC é: {imc:.2f}")
print(f"Classificação da OMS: {classificacao}")
```

- explicação do código:

O código utilizou a linguagem Python para resolução do problema. Criou duas funções, uma calculando de fato o IMC a partir do peso e da altura e, a outra, classificando o resultado do IMC de acordo com as normas da OMS. O código solicita do usuário o sexo, o peso (em Kg) e a altura em metros. Após a solicitação, executa as funções declaradas com base nos valores informados pelo usuário. Após, imprime na tela o sexo da pessoa, o resultado do IMC e a classificação segundo a OMS.

- pelo menos dois testes realizados;

Teste 1:

```
=== Calculadora de IMC (Padrão OMS) ===
Digite o sexo (M/F): M
Digite o peso em kg (ex: 70.5): 70
Digite a altura em metros (ex: 1.75): 1.75

--- Resultados ---
Sexo: Masculino
Seu IMC é: 22.86
Classificação da OMS: Peso normal
```

Teste 2:

```
Digite o sexo (M/F): A
Digite o peso em kg (ex: 70.5): -50
Digite a altura em metros (ex: 1.75): 1.60

--- Resultados ---
Sexo: Não especificado ou outro
Seu IMC é: -19.53
Classificação da OMS: Abaixo do peso
```

- análise crítica da solução produzida.

A solução foi adequada para o proposto, calculou o IMC de acordo com o cálculo correto e imprimiu os dados corretamente. Todavia, nota-se que o código não prevê casos não esperados, como, por exemplo, pesos negativos, ou alturas negativas, mostrando-se ineficiente para determinadas situações. Apesar de o sexo do usuário ser inútil para o cálculo, o código produzido se manteve fiel ao solicitado por mim.

Parte 4 – Melhorando o prompt

Modifique o prompt anterior para tornar a resposta mais útil. Inclua pelo menos três elementos adicionais, como:

- linguagem de programação;
- nível de explicação;
- comentários;
- exemplos de entrada e saída;
- tratamento de erros;

- restrições de implementação.

Prompt:

“crie um programa em C que calcule o Índice de Massa Corporal (IMC) de uma pessoa utilizando os criterios de classificacao da OMS.

O programa deve:

- Solicitar ao usuario o sexo (M ou m para masculino ou F ou f para feminino, o peso em quilogramas e a altura em metros)
- Calcular o IMC corretamente
- Exibir o valor do IMC com duas casas decimais
- Realizar tratamento de erros para impedir pesos ou alturas menores ou igual a 0 e para validar o sexo informado
- Não utilizar bibliotecas externa além das bibliotecas padrão da linguagem C

Nível da explicacao: Detalhado. Explique o funcionamento do algoritmo e das estruturas utilizadas

Restrições de implementacao:

- utilizar variaveis do tipo float ou double
- utilizar estruturas condicionais (if/else) para classificacao
- nao utilizar vetores, arquivos ou alocao dinamica de memoria.”

Código:

```
#include <stdio.h>

int main() {
    // Declaração das variáveis
    char sexo;
    float peso, altura, imc;

    printf("=== Calculadora de IMC (Padrão OMS) ===\n\n");

    // 1. Entrada e Validação do Sexo
    do {
        printf("Digite o sexo (M para Masculino, F para Feminino): ");
        scanf(" %c", &sexo); // O espaço antes do %c ignora quebras de linha residuais no buffer

        if (sexo != 'M' && sexo != 'm' && sexo != 'F' && sexo != 'f') {
            printf("Erro: Sexo invalido. Por favor, digite M ou F.\n");
        }
    } while (sexo != 'M' && sexo != 'm' && sexo != 'F' && sexo != 'f');

    // 2. Entrada e Validação do Peso
    do {
        printf("Digite o peso em kg (ex: 70.5): ");
        scanf("%f", &peso);

        if (peso <= 0) {
            printf("Erro: O peso deve ser maior que zero.\n");
        }
    } while (peso <= 0);

    // 3. Entrada e Validação da Altura
    do {
        printf("Digite a altura em metros (ex: 1.75): ");
        scanf("%f", &altura);

        if (altura <= 0) {
            printf("Erro: A altura deve ser maior que zero.\n");
        }
    } while (altura <= 0);

    // 4. Cálculo do IMC
    imc = peso / (altura * altura);

    // 5. Exibição e Classificação
    printf("\n--- Resultados ---\n");
    printf("Sexo informado: %c\n", sexo);
    printf("Seu IMC e: %.2f\n", imc);
    printf("Classificacao da OMS: ");

    // Estruturas condicionais para a tabela da OMS
    if (imc < 18.5) {
        printf("Abaixo do peso\n");
    } else if (imc >= 18.5 && imc < 25.0) {
        printf("Peso normal\n");
    } else if (imc >= 25.0 && imc < 30.0) {
        printf("Sobrepeso\n");
    } else if (imc >= 30.0 && imc < 35.0) {
        printf("Obesidade Grau I\n");
    } else if (imc >= 35.0 && imc < 40.0) {
        printf("Obesidade Grau II\n");
    } else {
        printf("Obesidade Grau III\n");
    }

    return 0;
}
```

Teste 1:

```
=== Calculadora de IMC (Padrão OMS) ===

Digite o sexo (M para Masculino, F para Feminino): a
Erro: Sexo invalido. Por favor, digite M ou F.
Digite o sexo (M para Masculino, F para Feminino): b
Erro: Sexo invalido. Por favor, digite M ou F.
Digite o sexo (M para Masculino, F para Feminino): c
Erro: Sexo invalido. Por favor, digite M ou F.
Digite o sexo (M para Masculino, F para Feminino): f
Digite o peso em kg (ex: 70.5): -5
Erro: O peso deve ser maior que zero.
Digite o peso em kg (ex: 70.5): -10
Erro: O peso deve ser maior que zero.
Digite o peso em kg (ex: 70.5): 0
Erro: O peso deve ser maior que zero.
Digite o peso em kg (ex: 70.5): 60
Digite a altura em metros (ex: 1.75): -5
Erro: A altura deve ser maior que zero.
Digite a altura em metros (ex: 1.75): 1.6

--- Resultados ---
Sexo informado: f
Seu IMC e: 23.44
Classificacao da OMS: Peso normal
PS C:\Users\andre> █
```

Compare a nova resposta com a versão anterior.

O código 2 demonstrou-se mais eficiente, tendo em vista que considerou tratamento de erros, além de que respeitou todas as condições propostas pelo meu prompt. Apesar de ambos terem sido parecidos (provavelmente pelo fato de a dificuldade de criação ser baixa), o resultado de ambos foi satisfatório.

Parte 5 – Verificação crítica

Escolha uma resposta da LLM e responda:

Escolhi a resposta do primeiro prompt do problema de cálculo de IMC

1. A resposta estava correta?

Estava correta, porém apresentava uma pequena falha.

2. Havia informações incompletas ou incorretas?

Não havia informações incorretas/incompletas, mas havia possíveis erros de execução e resultados (como IMCs negativos), tendo em vista que não havia tratamento de erros.

3. Como a resposta foi verificada?

A resposta foi verificada por meio de testes meus com o código gerado, pensando em possíveis execuções de usuários.

4. A LLM demonstrou excesso de confiança?

Acredito que sim, pois além do código, ela emitiu um texto que continha a seguinte frase: “[..] Ainda assim, incluí a variável no programa para atender fielmente à sua especificação! [...]”, demonstrando que foi completamente fiel ao que eu emiti no prompt, mas não verificou possíveis erros de execução (tratamento de erros).

5. Como o prompt poderia ser melhorado?

O prompt poderia ser melhorado levando-se em consideração tratamento de erros e melhores especificações do que exatamente eu queria (como a linguagem de programação a ser utilizada, por exemplo).

Parte 6 – Boas práticas de uso Inclua uma seção chamada 'Uso responsável de LLMs' respondendo:

1. Por que não devemos copiar respostas sem revisão?

Não deve-se copiar respostas sem revisão, principalmente porque nem sempre fica 100% claro exatamente aquilo que desejamos da IA, além do que, muitos casos de execução podem não ser levados em consideração (como no exemplo acima), fazendo com que seja necessária sempre uma revisão humana.

2. Por que é importante informar os prompts utilizados?

Para que quando uma revisão seja feita, seja possível entender o contexto que gerou determinada resposta, além de garantir transparência com o que foi proposto.

3. Em quais situações as LLMs ajudam no aprendizado?

Principalmente para aprender temas complexos, pois as IAs podem facilmente simplificá-las em analogias, por exemplo. Além disso, elas fornecem diferentes ferramentas para o aprendizado, como fazer resumos, flashcards etc.

4. Em quais situações elas podem atrapalhar?

Quando terceirizamos nosso aprendizado para elas, isto é, passamos a não pensar mais e a IA acaba nos substituindo quando não deveria. Mais além, a IA pode alucinar, comprometendo diretamente nosso uso.

5. O que significa utilizar uma LLM como apoio e não como substituta do raciocínio?

Significa utilizá-la como ferramenta auxiliar e não como nosso cérebro, ou seja, potencializamos nossa capacidade com o auxílio das inteligências artificiais.

Reflexão sobre o uso de IA Generativa

1. O que você aprendeu sobre interação com LLMs?

Apreendi que é necessário estruturar bem os prompts a serem utilizados objetivando respostas mais precisas e eficientes. Lembrando sempre que IAs podem gerar erros, logo, é sempre necessária a revisão humana acima de tudo!

2. Como a qualidade do prompt influenciou os resultados?

Dependendo do nível do prompt, o resultado era mais ou menos eficiente. Quanto mais detalhado e com mais informações, melhor o prompt. Quanto mais simples o prompt, menos controlado e impreciso o resultado sai.

3. Em quais situações a ferramenta foi mais útil?

Na criação de programas com especificações mais detalhadas, tendo resultados eficientes e de acordo com o prompt.

4. Em quais situações foi necessário verificar ou corrigir respostas?

Em todas as situações foi necessário verificar as respostas, visando identificar se a resposta era coerente com o prompt. Foi necessário corrigir a resposta do programa do IMC, levando em consideração o tratamento de erros.

5. Você utilizaria essa ferramenta em disciplinas futuras? Por quê?

Com absoluta certeza! Entendo que as IAs devem ser utilizadas como ferramentas para potencializar o aprendizado, sendo úteis em diversas situações. Todavia, noto que é importante ter cautela para não substituir o processo de desenvolvimento e o pensamento crítico.